How Far Are We from a Native Autoregressive Video Model?

Weiyang Jin, The University of Hong Kong · 31 August 2026

In language, the meaning of autoregression has been clear for a long time. Pretraining fits

$$p(w_{1:n}) = \prod_i p(w_i \mid w_{<i}),$$

and generating each next word takes one forward pass. During both training and inference, the model sees only words that have already been written. Nobody pretrains a bidirectional BERT [6], distills a student that "looks like GPT," and calls it a native autoregressive model. Adding a causal mask only at inference does not make it native either.

For video, native autoregression should mean the analogous factorization. For $V = (v_1, \ldots, v_T)$,

$$p(V \mid c) = \prod_t p(v_t \mid v_{<t}, c_{\le t}).$$

c_t denotes the conditioning information available at time t: text, a first frame, a camera pose, or a key press. These conditions may guide generation, but they do not give the model access to the future frames $v_{>t}$, which have not occurred yet. Here, native autoregression means two things: pretraining fits this product, and generating the next step takes one forward pass. Echo-WM-Flash [1] is one model we trained with a diffusion-autoregressive stack that is now common. The rest of this note asks how far that stack is from this definition.

An image has no time; video adds t

The left and right sides of an image share the same time index. PixelRNN [4] wrote $p(x) = \prod_{h,w} p(x_{hw} \mid x_{<hw})$ to impose an order on the joint distribution. That raster order is a coding convention, not a temporal constraint. DiT [12] instead uses bidirectional attention over (h, w), which is natural within a single frame.

Video adds a temporal axis t. Video Pixel Networks [5] already modeled that axis as a sequence at the pixel level rather than stretching duration T into a taller image. When v_t is generated, v_{t+1} does not yet exist—not because the current picture is unfinished, but because the next frame has not occurred. The spatial and temporal orderings therefore play different roles in the likelihood:

$$p(V) = \prod_t p\big(v_t \mid v_{<t}\big), \qquad p(v_t \mid v_{<t}) \text{ may be bidirectional on } (h, w).$$

The temporal product reflects the fact that later frames do not yet exist, whereas bidirectionality within a frame concerns only how the current frame is represented. Treating T as a longer H conflates "has not happened" with "the other side of the picture is unfinished." This distinction is easy to miss on clips that last only a few seconds: future frames are present in the same training example, and using them improves visual quality. Models such as Sora [14], Wan [16], and LTX [15] fit a joint distribution over an entire short clip, so their conditional at each time is

$$q_{\text{bi}}(v_t \mid V_{\setminus t}, c).$$

Streaming generation requires a different conditional distribution. The model must continue beyond its training window while accepting new conditions c_t as they arrive. It must therefore fit

$$q_{\text{AR}}(v_t \mid v_{<t}, c_{\le t}) = p_{\text{data}}(v_t \mid v_{<t}, c_{\le t}).$$



On a short clip, writing frame 5, 1–8 are all legal.



Streaming generation only admits the past. Writing frame 5, 6–8 do not exist.

The conditional distribution a model learns depends on which variables it can see during training. Changing c_t from text to an image or pose does not remove access to future frames: the temporal context remains $V_{\setminus t}$, not $v_{<t}$. Interactive world models [22–25] exposed this mismatch early because user actions arrive while generation is underway. Long-video generation and low-latency continuation encounter the same problem.

VideoGPT [7] and TATS [9] already modeled discrete video tokens with $\prod_t p(v_t \mid v_{<t})$. As masked-token models such as MaskGIT [8], MAGVIT [11], and Phenaki [10] improved short-clip quality, autoregressive token models fell behind; bidirectional DiT-scale models widened the gap further. Visual quality improved, but past-only temporal pretraining was abandoned. When streaming generation became important, work focused not on restoring that pretraining objective, but on adapting an existing q_{bi} so that inference resembles q_{AR}. The pipeline described below grew out of that effort.

Training sees the future, and the next step takes more than one forward

By that definition, the diffusion-autoregressive stack now in use satisfies neither condition.

Consider training first. The target should be $q_{\text{train}}(v_t \mid \cdot) = p_{\text{data}}(v_t \mid v_{<t})$. If pretraining instead fits q_{bi}, adding a causal mask at inference—so that the current block cannot see later blocks—only hides information that was available during training. The model was never trained under a past-only condition.

Now consider generation. After dividing time into $B_1, \ldots, B_N$, the model is described by $p(B_{1:N}) = \prod_i p_\theta(B_i \mid B_{<i})$. But if each step is implemented as

$$x_{\sigma_K} \sim \mathcal{N}(0, I), \quad x_{\sigma_k} = x_{\sigma_{k+1}} + (\sigma_k - \sigma_{k+1})\, v_\theta(x_{\sigma_{k+1}}, \sigma_{k+1} \mid B_{<i}, B_i^{\text{noisy}}),$$

then B_i is obtained by integrating an ordinary differential equation rather than by a single network evaluation. Sampling from the bidirectional short-clip model may require $K \sim 30$ integration steps. Reducing the number of steps to $K = 4$ shortens the integration, but it still does not produce $f_\theta(v_{<t})$ in one forward pass.

These two mismatches lead to a chain of post-training stages. Because pretraining used future frames, inference first adds a causal mask. A second mismatch then appears: training conditions on clean data through teacher forcing, whereas inference conditions on blocks the model generated itself:

$$q_{\text{train}}(B_{<i}) = p_{\text{data}}(B_{<i}), \qquad q_{\text{infer}}(B_{<i}) = p_\theta(B_{<i}).$$

Scheduled sampling [2], sequence-level training [3], and Self Forcing [19] all address this train–test mismatch. Rather than replace bidirectional pretraining with a genuinely past-only objective, the stack adds another stage: first train with DMD [28] on histories rolled out by the model, then apply the same loss to longer histories that already contain errors, $\mathcal{L}_{\text{long}} = \sum_k \mathcal{L}_{\text{DMD}}^{(k)}$. These stages compensate after the fact for a condition that pretraining never used.

Generating the next step still requires integrating a differential equation, which introduces another distillation stage. Diffusion Forcing [17], CausVid [18], Self Forcing [19], Rolling Forcing [20], and MAGI-1 [21] avoid future frames at inference but still use several denoising steps within each block. A full generation therefore takes $N_{\text{blocks}} \times K$ forward passes. Even at $K = 4$, the network still predicts the vector field $v_\theta(x, \sigma)$.

The pretraining condition remains unchanged, and each generated step remains a multi-step integral. To generate beyond the training window, systems then modify the visible history $\mathcal{H}_i$: retain a few early frames as anchors, keep only the most recent blocks, or reset positional encodings as the window advances. StreamingLLM [13] showed why a sliding language-model window needs anchors; FramePack [26] and WorldMem [27] ask what $\mathcal{H}_i$ should retain for video. These choices change which frames are visible at inference, not the condition fitted during training.

Together, these corrections produce the stack now in common use:

CURRENT STACK

```
for i in 1..N:
    x ~ N(0, I)
```

NATIVE

```
for t in 1..T:
    v_t = f_θ(v_<t, c_≤t)
```

```
    for σ in [1.0, 0.75, 0.50, 0.25]:
        x = x + Δσ · v_θ(x, σ | B_<i, x)
    cache.write(x)
```

```
    cache.write(v_t)
```

PRETRAIN	bidirectional DiT, full short clip Fits q_bi, not p(v_t | v_<t).
CAUSAL	block-causal mask, clean history Hides at test time the future training used. History is still data.
SHORT	self-rollout + DMD Patches K, and the teacher-forcing gap. History is still short.
LONG	stitch + fixed window Patches length. The training condition is unchanged.
SEARCH	window size, whether to reset position Edits H_i. Still not parameters.

Each extra stage introduces more quantities to tune. If pretraining excluded $v_{>t}$ and generation required one forward pass per step, these stages would be unnecessary.

Block length is therefore not a free choice. A one-frame block contains no intra-block motion, so additional integration steps operate only within that frame. A block approaching one second turns the system back into a short-clip model with a cache at each boundary. In the stack considered here, three frames became the working compromise: they are short enough for incremental output but long enough for bidirectional denoising within the block. A native model must choose its temporal unit during pretraining—one forward pass per frame or per block—rather than grouping three frames under one mask and then integrating within that group.

In our experience

We have trained models with this stack; Flash [1] is one example. Two patterns recurred during closed-loop, long-horizon generation.

If post-training optimizes denoising inside a block, B_i can look better without improving the conditional dependence between blocks. That cross-block dependence is part of what $\prod_t p(v_t \mid v_{<t})$ requires. Changing only $\mathcal{H}_i$ alters which frames are visible at inference, not the condition fitted during training.

	APPEARANCE INSIDE A BLOCK	CONSISTENCY ACROSS BLOCKS
Later post-training	**Cleaner in the same block**	No reliable gain
Inference window / reset only	—	No reliable gain

With the sampler and context window fixed, additional post-training stages produced cleaner textures within the same block B_i, but consistency across blocks did not improve reliably.

We also held the generator fixed and changed only $\mathcal{H}_i$, either by lengthening the window or by switching positional and pose resets on or off. A window that differs from the one used in training introduces discrepancies at block boundaries; a

different reset policy makes relative positional encodings describe a different geometry. Neither change reliably improved consistency across blocks. They altered which frames were visible at inference without fitting $p(v_t \mid v_{<t})$.

What that means for a native design

Neither intra-block post-training nor changes to the inference window or reset policy can fit $p(v_t \mid v_{<t})$. If pretraining must fit $p(v_t \mid v_{<t}, c_{\le t})$ and each next step must take one forward pass, several design choices follow.

Post-training inside a block does not supply the condition between blocks. An intra-block loss focuses model capacity on the appearance of B_i. Learning $\prod_t p(v_t \mid v_{<t})$ instead requires pretraining on past frames alone; another distillation stage that sharpens textures cannot provide that condition. Continued pretraining can preserve the initialization of a bidirectional DiT, but future frames must be hidden from the first continued-pretraining update. Starting from random initialization is conceptually cleaner: retain the same latent space and bidirectional attention within each frame, but use a causal temporal mask from the beginning. VideoGPT [7] already trained discrete video tokens this way, although its codebook limited visual quality. The missing piece is to use the same objective with a DiT and a modern video VAE [12 15 16]. The loss may remain flow matching; only the condition changes:

$$\mathcal{L} = \mathbb{E}_{t,\sigma} \left\| v_\theta(x_{\sigma,t}, \sigma \mid v_{<t}, c_{\le t}) - (\varepsilon - v_t) \right\|_2^2,$$

Here, σ is applied only to the current unit, and $v_{>t}$ is excluded from the computation graph. To meet the native criterion at $K = 1$, consistency training must use q_{AR} from the start. Learning q_{bi} first and compressing the step count later again devotes capacity to integration within the block. Post-training can improve that integration, but the improvement does not reliably transfer to consistency across blocks. Self-rollout followed by DMD can continue to compensate for the gap, but it remains post-training rather than native autoregressive pretraining.

A longer window, or turning a reset on and off, does not change the training condition. Altering $\mathcal{H}_i$ without changing that condition does not reliably improve consistency across blocks. In a native model, the visible past must therefore be represented by the model itself: either directly as $v_{<t}$ in a causal-attention cache, or through a learned, end-to-end compression of $v_{<t}$. Manually choosing the numbers of anchor frames and recent frames, then sweeping the positional-reset policy, is necessary only when adapting q_{bi} to resemble q_{AR}. FramePack [26] and WorldMem [27] ask what this history should retain. Another inference window that differs from training does not answer that question.

Choose the temporal unit during pretraining; do not integrate within it at inference. An intra-block denoiser may improve appearance while still requiring K-step integration of the vector field. If one frame or block is defined as one autoregressive step, applying K denoising steps to that unit would violate the one-forward-pass requirement. One forward pass per frame is the closest analogue to one word in language. A single pass may instead generate $L > 1$ frames with bidirectional attention within the block, producing a longer segment at each step. Both choices are valid, but L cannot remain an inference-time hyperparameter because changing it changes the definition of a step. If block length remains an inference-time compromise, the system returns to a short block with an inner differential-equation solver, which post-training must optimize again.

Measure consistency across blocks, not only appearance within them. A better-looking block is not necessarily more consistent with the blocks around it. Evaluation based only on short clips and intra-block metrics conflates the two. Use one set of weights, generate each step in one forward pass, and evaluate consistency beginning in the first second and continuing beyond the training window. When c_t changes mid-stream, check whether the video context remains limited to $v_{<t}$ while the new condition takes effect. A bidirectional model may look better on clips lasting a few seconds because it saw future frames at that length during training. Short-clip appearance alone therefore says little about whether native autoregression works.

Construct training data around what is available at each time. Fixed-length clips make future frames available as features. Training q_{AR} instead requires longer continuous shots, timestamped c_t, and conditions that first appear in the mid-

dle of a sequence. Changes that leave the training condition untouched do not reliably improve consistency across blocks, as we observed when changing only $\mathcal{H}_i$.

A few questions remain open and should be answered before another window search. Can time-causal continued pretraining erase the behavior learned by a large bidirectional video backbone, and if so, how long does it take? Or is reinitialization necessary? Is $K = 1$ stable in a continuous latent space, or does it require a stronger discrete or hybrid codebook? Under a native objective, should each forward pass generate one frame or three to balance motion quality and latency? Should consistency across blocks directly supervise a learned $\mathcal{H}_i$? Searching over $\mathcal{H}_i$ without that supervision does not change the condition learned during training.

The move away from VideoGPT-style next-frame prediction made sense in 2021, when the goal was to generate short clips and q_{bi} was well suited to that setting. The task now includes streaming generation. Pretraining a bidirectional clip model, adding post-training, and searching over $\mathcal{H}_i$ at inference still does not fit $p(v_t \mid v_{<t})$. That condition must change during pretraining. Until it does, systems now called “autoregressive video” still begin by denoising whole short clips and restrict attention to the past only at inference.

Conclusion

Native autoregression has two requirements: pretraining fits $p(v_t \mid v_{<t}, c_{\leq t})$, and generating the next step takes one forward pass. The current stack satisfies neither; post-training cannot add either property retroactively. An intra-block loss can improve the appearance of a block, while a longer window or a reset changes only which frames are visible at inference. Neither changes the pretraining objective to $p(v_t \mid v_{<t})$.

Language models scaled rapidly while retaining next-token prediction as their pretraining objective. Beginning around 2021, video generation moved away from next-frame prediction in pursuit of better short clips. Causal masks, few-step denoising, and window search were added later to make bidirectional clip models work in a streaming setting, but none of them changes the condition learned during pretraining. A native model must instead exclude future frames during pretraining and produce each next step in a single forward pass. Until both changes are made, we are still far from a native autoregressive video model.

References

1. Zhang, S., Li, Y., Zhuang, J., Jin, W., et al. *EchoWM: Open and Enterable Omnimodal World Models.* arXiv:2608.23189, 2026. https://arxiv.org/abs/2608.23189
2. Bengio, S., Vinyals, O., Jaitly, N., and Shazeer, N. *Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks.* NeurIPS, 2015.
3. Ranzato, M., Chopra, S., Auli, M., and Zaremba, W. *Sequence Level Training with Recurrent Neural Networks.* ICLR, 2016. https://arxiv.org/abs/1511.06732
4. van den Oord, A., Kalchbrenner, N., and Kavukcuoglu, K. *Pixel Recurrent Neural Networks.* ICML, 2016. https://arxiv.org/abs/1601.06759
5. Kalchbrenner, N., van den Oord, A., Simonyan, K., et al. *Video Pixel Networks.* 2016. https://arxiv.org/abs/1610.00527
6. Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.* NAACL, 2019.
7. Yan, W., Zhang, Y., Abbeel, P., and Srinivas, A. *VideoGPT: Video Generation using VQ-VAE and Transformers.* 2021. https://arxiv.org/abs/2104.10157
8. Chang, H., Zhang, H., Jiang, L., Liu, C., and Freeman, W. T. *MaskGIT: Masked Generative Image Transformer.* CVPR, 2022. https://arxiv.org/abs/2202.04200
9. Ge, S., Hayes, T., Yang, H., et al. *Long Video Generation with Time-Agnostic VQGAN and Time-Sensitive Transformer.* ECCV, 2022. https://arxiv.org/abs/2204.03638
10. Villegas, R., Babaeizadeh, M., Kindermans, P.-J., et al. *Phenaki: Variable Length Video Generation from Open Domain Textual Descriptions.* 2022. https://arxiv.org/abs/2210.02399
11. Yu, L., Cheng, Y., Sohn, K., et al. *MAGVIT: Masked Generative Video Transformer.* ICLR, 2023. https://arxiv.org/abs/2212.05199
12. Peebles, W., and Xie, S. *Scalable Diffusion Models with Transformers.* ICCV, 2023. https://arxiv.org/abs/2212.09748
13. Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. *Efficient Streaming Language Models with Attention Sinks.* ICLR, 2024. https://arxiv.org/abs/2309.17453
14. OpenAI. *Video generation models as world simulators (Sora).* 2024. https://openai.com/index/video-generation-models-as-world-simulators/
15. HaCohen, Y., Chiprut, N., Brazowski, B., et al. *LTX-Video: Realtime Video Latent Diffusion.* 2024. https://arxiv.org/abs/2501.00103
    Lightricks. *LTX-2 / LTX-2.3.* 2025–2026.
16. Wan Team. *Wan: Open and Advanced Large-Scale Video Generative Models.* 2025. https://arxiv.org/abs/2503.20314

17. Chen, B., Martí Monsó, D., Du, Y., et al. *Diffusion Forcing: Next-token Prediction Meets Full-Sequence Diffusion.* 2024. https://arxiv.org/abs/2407.01392
18. Yin, T., Zhang, Q., Zhang, R., Freeman, W. T., Durand, F., Shechtman, E., and Huang, X. *From Slow Bidirectional to Fast Autoregressive Video Diffusion Models (CausVid).* 2024. https://arxiv.org/abs/2412.07772
19. Huang, X., Li, Z., He, G., Zhou, M., and Shechtman, E. *Self Forcing: Bridging the Train-Test Gap in Autoregressive Video Diffusion.* NeurIPS, 2025. https://arxiv.org/abs/2506.08009
20. Liu, K., Hu, W., Xu, J., Shan, Y., and Lu, S. *Rolling Forcing: Autoregressive Long Video Diffusion in Real Time.* 2025. https://arxiv.org/abs/2509.25161
21. Teng, H., Jia, H., Sun, L., et al. *MAGI-1: Autoregressive Video Generation at Scale.* Sand AI, 2025. https://arxiv.org/abs/2505.13211
22. Bruce, J., Dennis, M., Edwards, A., et al. *Genie: Generative Interactive Environments.* ICML, 2024. https://arxiv.org/abs/2402.15391
23. NVIDIA. *Cosmos World Foundation Model Platform for Physical AI.* 2025. https://arxiv.org/abs/2501.03575
24. Gao, Z., Wang, Q., Zhu, J., et al. *Infinite Worlds with Versatile Interactions (LingBot-World 2.0).* 2026. https://arxiv.org/abs/2607.07534
25. Zhu, H., Liu, H., Zhao, Y., et al. *SANA-WM: Efficient Minute-Scale World Modeling with Hybrid Linear Diffusion Transformer.* 2026. https://arxiv.org/abs/2605.15178
26. Zhang, L. *Packing Input Frame Context in Next-Frame Prediction Models (FramePack).* 2025. https://github.com/lllyasviel/FramePack
27. Xiao, Z., Lan, Y., Zhou, Y., Ouyang, W., Yang, S., Zeng, Y., and Pan, X. *WorldMem: Long-term Consistent World Simulation with Memory.* NeurIPS, 2025. https://arxiv.org/abs/2504.12369
28. Yin, T., Gharbi, M., Zhang, R., Shechtman, E., Durand, F., Freeman, W. T., and Park, T. *One-step Diffusion with Distribution Matching Distillation.* CVPR, 2024. https://arxiv.org/abs/2311.18828

Cite

```
@misc{jin2026nativear,
  author       = {Jin, Weiyang},
  title        = {How Far Are We from a Native Autoregressive Video Model?},
  year         = {2026},
  month        = aug,
  institution  = {The University of Hong Kong},
  howpublished = {Blog post},
  url          = {https://waynejin0918.github.io/how-far-native-ar-video/},
}
```

Visual style after Thinking Machines Lab.